

vue.js
AUG 12

第二章 Vue.js开发基础

第2章 Vue开发基础

■ Vue应用实例

- 模板语法（插值、指令）

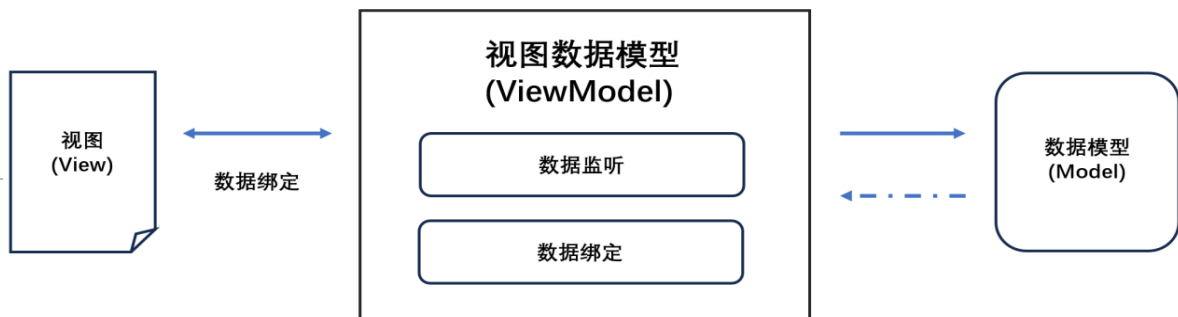
- 方法属性

- 属性绑定

- 插入html片段

■ Vue实例的生命周期

2.1 Vue实例



Vue.js遵循MVVM模式：

核心工作—创建一个“视图模型” (ViewModel)对象，作为“视图” (View)和“业务模型” (Model)之间的桥梁。

- Vue.js的做法是提供一个**Vue类型**，开发者通过创建一个**Vue类型的实例**，来实现**视图模型的定义**。
- 在一个完整的项目中，可能会创建出多个Vue的实例，形成层次结构，但是通常一个应用中会有一个唯一的最上层的Vue实例，这个实例就称为**“根实例”**。
- 具体的语法形式如下所示。

```
const vm = new Vue({  
  // 选项对象  
})
```

2.1 Vue实例

```
const vm = new Vue({  
  // 选项对象  
})
```

JS角度：

- Vue是一个类型，Vue()是它的构造函数。
- 用new调用Vue()构造函数创建一个“实例” / “对象”。
- 调用构造函数时，传入一个参数(JavaScript对象方式)。这个对象中可以设定很多选项，这些选项指定这个实例的行为，即它与页面如何配合工作。
- Vue.js重点—设置对象。

Vue案例分析

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
<meta charset="UTF-8">
```

```
<title>初识Vue</title>
```

```
<script src="vue.js"></script>
```

<!--引入Vue -->

```
</head>
```

```
<body>
```

```
<div id="root">
```

```
<h1>Hello, 软件学院</h1>
```

```
</div>
```

<!--准备一个容器 -->

```
<script>
```

```
const x = new Vue({  
  el:'#root',  
})
```

//创建Vue实例

```
</script>
```

```
</body>
```

```
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>初识Vue</title>
  <!--引入Vue -->
  <script src="vue.js"></script>
</head>
<body>
  <!--准备一个容器 -->
  <div id="root">
    <h1>Hello, {{name}}</h1>
  </div>

  <script>
    const x = new Vue({
      el:'#root',
      data:{
        name:'网络安全'
      }
    })
  </script>
</body>
</html>
```

初识Vue

1. 如果想让Vue工作，就必须创建一个Vue实例，且要传入一个配置对象。
2. root容器里的代码依然符合HTML规范，只不过混入了一些特殊的Vue语法
3. root容器里的代码被称为【Vue模板】。

插值语法

//el用于指定当前Vue实例为哪个容器服务，值通常为CSS选择器字符串

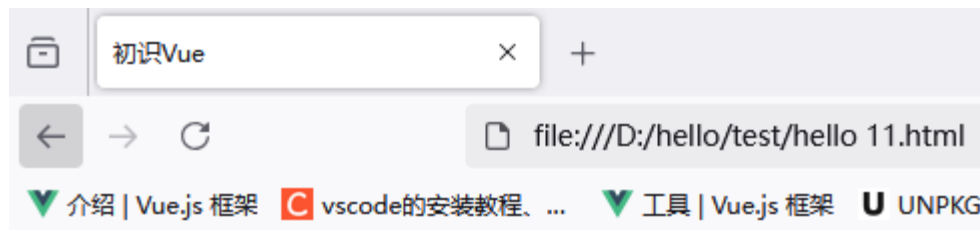
//data中用于存储数据，数据供el所指定的容器使用，指我们暂且先写为一个对象

```
<script src="vue.js"></script>
</head>
<body>
```

```
<div class="root">
  <h1>Hello, {{name}}1</h1>
</div>
```

```
<div class="root">
  <h1>Hello, {{name}}2</h1>
</div>
```

```
<script >
  const x = new Vue({
    el: '.root',
    data:{
      name:'网络安全'
    }
  })
</script>
```



Hello,网络安全1

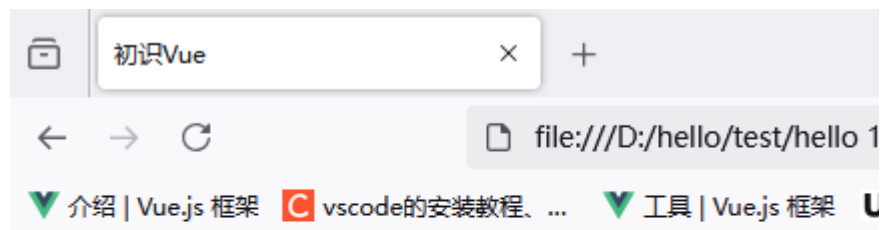
Hello,{{name}}2

```
<script src="vue.js"></script>
</head>
<body>
```

- 容器和实例是一一对应的!
- 真实开发中只有一个Vue实例，且会配合组件一起使用。

```
<div id="root">
  <h1>Hello, {{name}},{{age}}</h1>
</div>
```

```
<script>
  new Vue({
    el: '#root',
    data:{
      name:'网络安全'
    }
  })
  new Vue({
    el: '#root',
    data:{
      age:18
    }
  })
</script>
```



Hello,网络安全,

{{xxx}}中的xxx可以写什么?

<h1>Hello, {{name}},{{age}}</h1>

{{ }}内写JS表达式

注意区分：JS表达式和JS代码(语句)

1、**JS表达式**：一个表达式会产生一个值，
可以放在任何一个需要值的地方。

- ① a
- ② a+b
- ③ Demo (1)
- ④ x === y ? 'a' : 'b'

2、**JS代码(语句)**

- ① if(){}
- ② for(){}

- {{xxx}}中写JS表达式，而且xxx可以自动读取到data中的所有属性；
- 一旦data中数据发生改变，模板中用到该数据的地方也会自动更新。

模板语法

```
<a v-bind:href= "url" >点我去工大</a>
```

```
<body>
```

```
<div id="root">
```

```
<h1>插值语法</h1>
```

```
<h3>Hello,{{name}}</h3>
```

```
<hr/>
```

```
<h1>指令语法</h1>
```

```
<a href= "url" >点我去工大</a>
</div>
```

```
<script >
```

```
new Vue({
```

```
  el:'#root',
```

```
  data:{
```

```
    name:'annie',
```

```
    url:'http://www.tiangong.edu.cn'
```

```
  }
```

```
})
```

```
</script>
```

```
</body>
```

v-bind是Vue指令，用于数据绑定，绑定后“xxx”中的xxx被当作JS表达式处理。

```
<body>
  <div id="root">
    <h1>插值语法</h1>
    <h3>Hello,{{name}}</h3>
    <hr/>
    <h1>指令语法</h1>
    <a v-bind:href="school.url">点我去{{school.name}}</a>
  </div>
```

```
<script >
  new Vue({
    el:'#root',
    data:{
      name:'annie',
      school:{
        name:'工大',
        url:'http://www.tiangong.edu.cn',
      }
    }
  })
</script>
</body>
```

Vue模板语法有两大类：

1、插值语法

功能：用于解析标签体内容

写法：{{xxx}}，xxx是JS表达式。

2、指令语法

功能：用于解析标签（包括标签属性、标签内容、绑定事件.....）。

举例：v-bind:href= "xxx" 可以简写为:href= "xxx"，xxx同样要写JS表达式，且可以直接读取到data中的所有属性。

备注：Vue中有很多指令，且形式都是V-???。

数据绑定

```
<head>
  <meta charset="UTF-8" />
  <title>数据绑定</title>
  <script type="text/javascript"
src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js" > </script>
</head>
<body>
  <div id="root">
    数据绑定: <input type="text" :value="name" > <br/>
  </div>
</body>

<script type="text/javascript">
  new Vue({
    el:'#root',
    data:{
      name:'天工大'
    }
  })
</script>
```

数据绑定

```
<head>
  <meta charset="UTF-8" />
  <title>数据绑定</title>
  <script type="text/javascript"
src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"> </script>
</head>
<body>
  <div id="root">
    单向数据绑定: <input type="text" v-bind:value="name"> <br/>
    双向数据绑定: <input type="text" v-model:value="name">
  </div>
</body>

<script type="text/javascript">
  new Vue({
    el:'#root',
    data:{
      name:'天工大'
    }
  })
</script>
```

Vue中有两种数据绑定的方式:

- 1、**单向绑定**: 数据只能从data流向页面
- 2、**双向绑定**: 数据不仅能从data流向页面, 还可以从页面流向data。

备注:

- 1、双向绑定一般都应用在表单类元素上 (如input select等)。
- 2、V-model:value可以简写为V-model, 因为V-model默认收集的就是value值。

Vue3与Vue2

Vue3

```
// 导入 createApp (模块化环境)
import { createApp } from 'vue';
const { createApp } = Vue
// 创建应用实例
const app = createApp({
  // 选项对象 (data、methods、生命周期等)
  data() {
    return {
      message: 'Hello Vue 3!'
    };
  },
  template: `<h1>{{ message }}</h1>`
});

// 挂载到 DOM 元素
app.mount('#app');
```

```
new Vue({
  el: '#root',
  data: {
    name: '天工大'
  }
})
```

Data两种写法

1. data中所有的属性最后都出现在了VM上。
2. VM身上所有的属性以及Vue原型上所有的属性，在Vue模板中都可以直接使用。

数据代理

Object.defineProperty () 方法——直接定义或修改对象的属性，并允许开发者精确控制属性的行为（如是否可写、可枚举、可配置等）。

Object.defineProperty(obj, propName, descriptor)

- obj: 目标对象。
- propName: 属性名（字符串形式）。
- descriptor: 属性描述符对象，定义属性的特性。

```
<script type="text/javascript">
  let person = {
    name:'张三',
    sex:'男',
  }
  Object.defineProperty(person,'age',{
    value:18,
    enumerable:true, //控制属性是否可以枚举，默认False
    writable:true, //控制属性是否可以被修改，默认F
    configurable:true, //控制属性是否可以被删除，默认F

  })
  console.log(Object.keys(person))
  console.log(person)
</script>
```

数据代理

```
<script type="text/javascript">
  let number = 18
  let person = {
    name:'张三',
    sex:'男',
  }
  Object.defineProperty(person,'age',{
    //当有人读取person的age属性时，get函数（getter）就会被调用，
    //且返回值就是age的值
    get(){
      return number
    },
    //当有人修改person的age属性时，set函数（setter）就会被调用，
    //且会收到修改的具体值
    set(value){
      console.log('有人修改age属性，且值是',value)
      number=value
    }
  })
  console.log(Object.keys(person))
  console.log(person)
</script>
```


回顾

- 如何引入Vue，创建Vue实例
- Vue中的MVVM模型
- Vue模板语法
 - 插值语法
 - 指令语法——数据绑定
- 数据代理

数据代理

定义：通过一个对象代理对另一个对象中属性的操作（读/写）。

```
<script type="text/javascript">
```

```
let obj = {x:100}
```

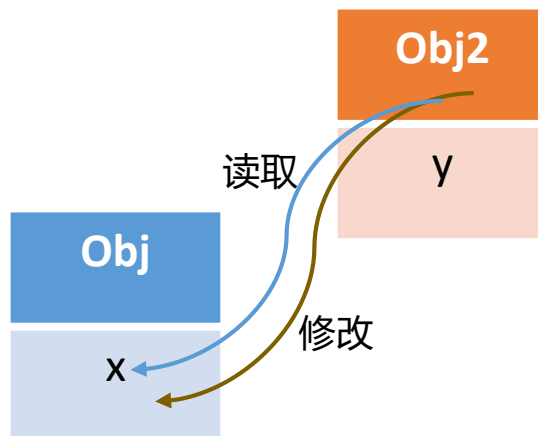
```
let obj2 = {y:200}
```

```
Object.defineProperty(obj2, 'x', {
```

```
  get(){  
    return obj.x  
  },
```

```
  set(value){  
    obj.x=value  
  }  
})
```

```
</script>
```

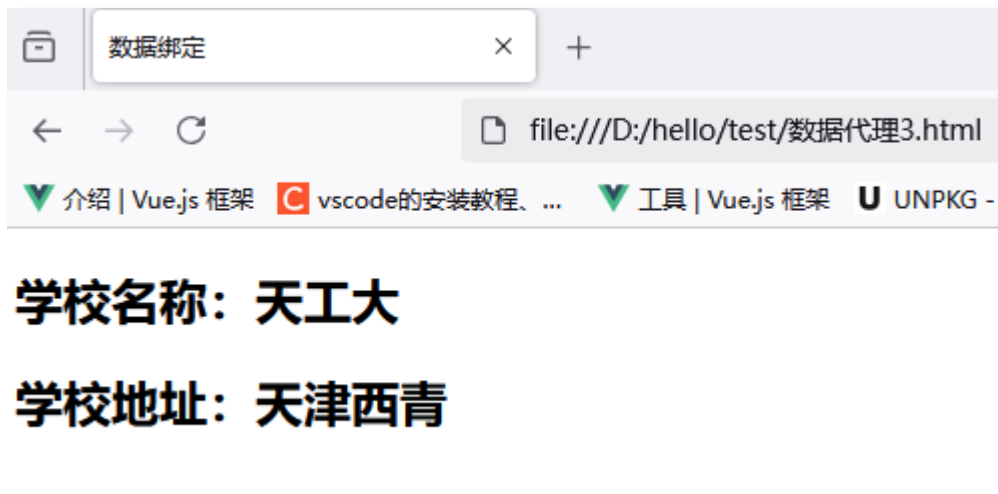


数据代理

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="UTF-8" />
    <title>数据代理</title>
    <script type="text/javascript"
src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"></script>
  </head>
  <body>
    <div id="root">
      <h2>学校名称: {{name}}</h2>
      <h2>学校地址: {{address}}</h2>
    </div>
  </body>

  <script type="text/javascript">
Vue.config.productionTip = false

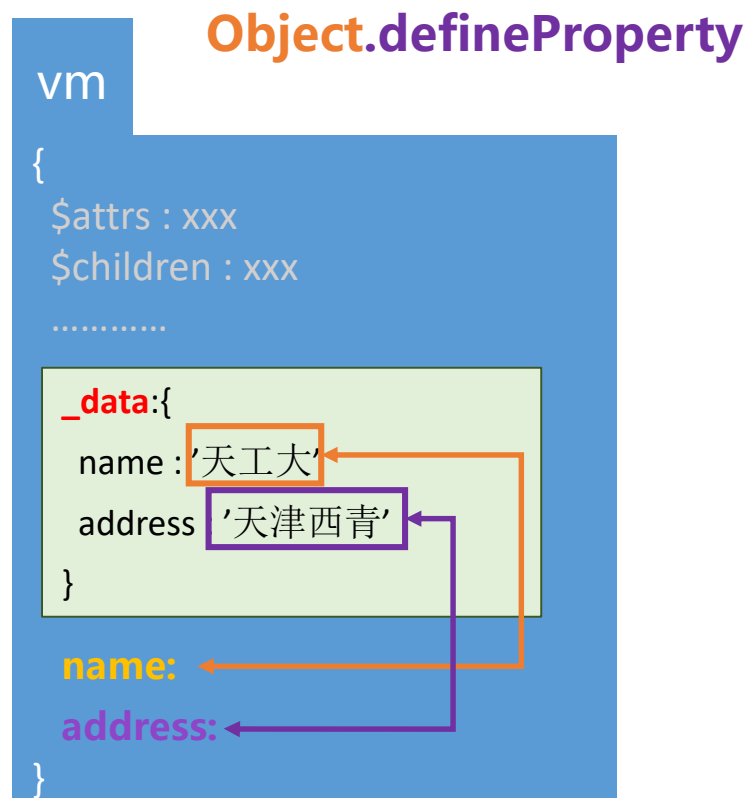
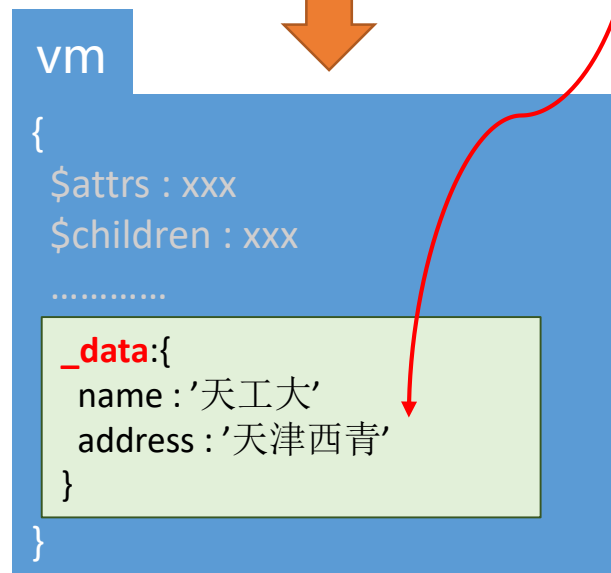
const vm = new Vue({
  el:'#root',
  data:{
    name:'天工大',
    address:'天津西青'
  }
})
</script>
</html>
```



Vue中数据代理

1. Vue中的数据代理：通过VM对象来代理data对象中属性的操作(读/写)
2. 好处：更加方便的操作data中的数据。
3. 基本原理：通过object.defineProperty把data对象中的所有属性添加VM上，为每一个添加到VM上的属性,都指定一个getter / setter，在getter/setter内部去操作(读/写)data中对应的属性。

```
const vm = new Vue({  
  el: '#root',  
  data: {  
    name: '天工大',  
    address: '天津西青'  
  }  
})
```

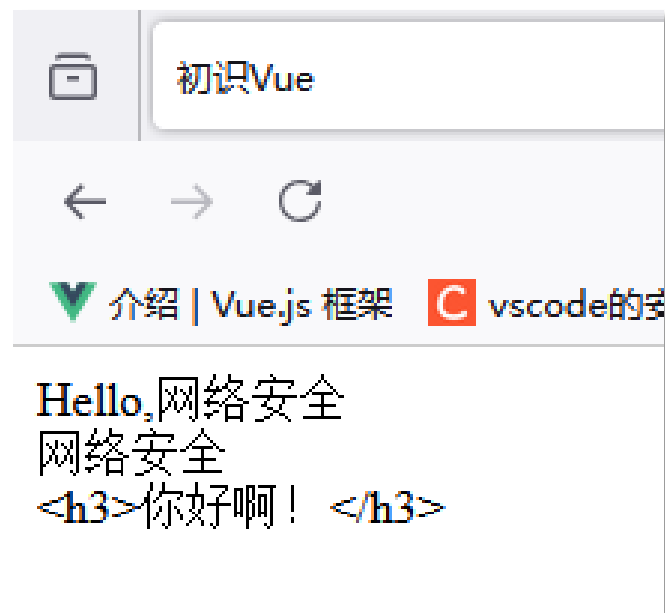


内置指令 v-text

```
<body>
  <div id="root">
    <div>Hello, {{name}}</div>
    <div v-text="name">Hello, </div>
    <div v-text="str"></div>

  </div>
</body>

<script type="text/javascript">
  new Vue({
    el:'#root',
    data:{
      name:'天工大'
      str: '<h3>你好啊! <h3>'
    }
  })
</script>
```



v-text:

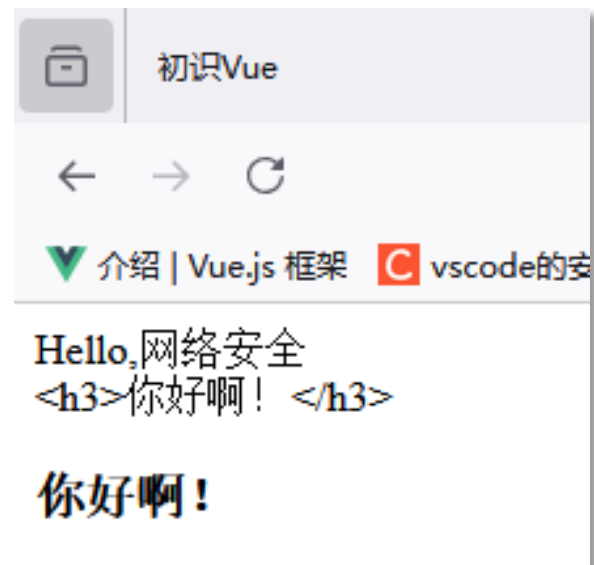
1. 作用：向其所在的节点中渲染文本内容。
2. 与插值语法区别：v-text会替换掉节点中的内容，{{xxx}}不会。

```
str2:'<a href=javascript:location.href="http://www.baidu.com?">document.cookie>'
```

内置指令 v-html

```
<body>  
  <div id="root">  
    <div>Hello, {{name}}</div>  
    <div v-text="str"></div>  
    <div v-html="str"></div>  
  </div>  
</body>
```

```
<script type="text/javascript">  
  new Vue({  
    el:'#root',  
    data:{  
      name:'天工大'  
      str: '<h3>你好啊! <h3>'  
    }  
  })  
</script>
```



v-html:

1. 作用：向指定节点中渲染包含html结构的内容。
2. 与插值语法区别：
 - ① v-html会替换掉节点中的内容，{{xxx}}不会。
 - ② v-html可以识别html结构。
3. v-html有安全性问题

内置指令 v-pre

```
<!-- 完整语法 -->  
<a v-on:click="doSomething">...</a>  
  
<!-- 缩写 -->  
<a @click="doSomething">...</a>
```

```
<body>  
  <!-- 准备一个容器 -->  
  <div id="root">  
    <h1 v-pre>Vue其实很简单</h1>  
    <h1 v-pre>当前n的值是,{{n}}</h1>  
    <button @click="n++">点我加n+1</button>  
  </div>
```

```
<script >  
  new Vue({  
    el: '#root',  
    data: {  
      n: 1  
    }  
  })  
</script>  
</body>
```

Vue其实很简单

当前的n值是:{{n}}

点我n+1

v-pre (没有值) :

1. 跳过其所在节点的编译过程。
2. 可跳过: 没有使用指令语法、插值语法的节点, 加快编译。

内置指令 v-cloak

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8" />
```

```
  <script type="text/javascript"
src="https://cdn.jsdelivr.net/npm/vue@2/dist/vue.js"> </script>
```

```
</head>
```

```
<body>
```

```
  <div id="root">
```

```
    <h2>{{name}}</h2>
```

```
  </div>
```

```
</body>
```

```
<script type="text/javascript">
Vue.config.productionTip = false
```

```
const vm = new Vue({
  el:'#root',
  data:{
    name:'天工大',
  }
})
```

```
</script>
</html>
```

```
<style>
[v-cloak]{
  display: none;
}
</style>
```

```
<h2 v-cloak>{{name}}</h2>
```

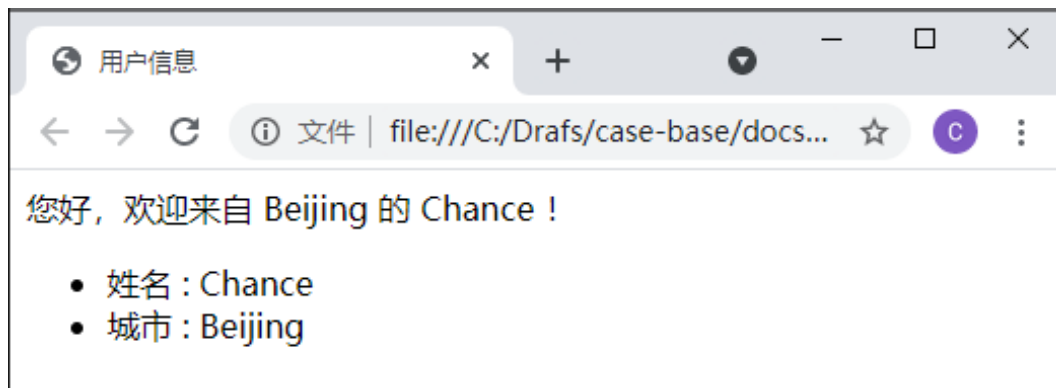
v-cloak (没有值) :

1. 特殊属性, Vue实例创建完毕并接管容器后, 会删掉v-cloak属性。
2. 使用css与v-cloak配合可以解决网速慢时, 显示{{xxx}}问题。

2.1 Vue实例

■ 方法属性

传入Vue()构造函数的对象中，除了`el`和`data`，还可以包括方法（`methods`）属性，里面可以定义多个方法（函数）。



例：页面上根据用户的姓名和所在城市，显示一句欢迎词“欢迎来自某地的某人”。——添加一个sayHello()函数，其作用是将name和city插入一个模板字符串中，得到一个完整的句子。

Vue实例-方法属性

```
<div id="app">
  <p>{{sayHello()}}</p>
  <ul>
    <li>姓名：{{name}}</li>
    <li>城市：{{city}}</li>
  </ul>
</div>
```

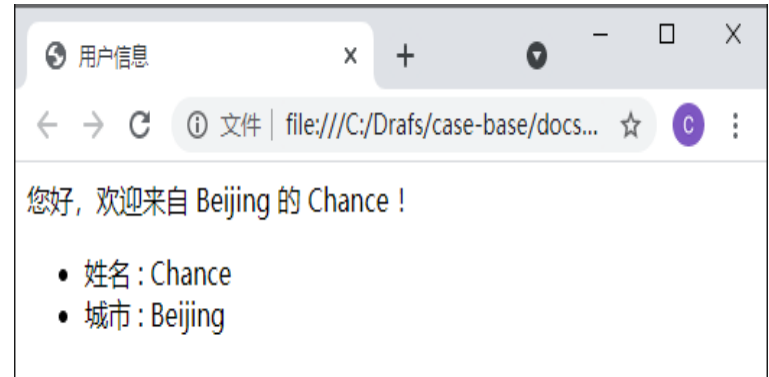
```
<script>
```

```
let user = {
  name: "Chance",
  city: "Beijing"
};

const { createApp } = Vue
const app = createApp({
  data() {
    return user
  },
```

```
  methods: {
    sayHello() {
      return `您好，欢迎来自 ${this.city} 的 ${this.name} !`
    }
  }
})
```

```
  }).mount('#app')
</script>
```



上下文绑定

ES6新增的语法结构——“字符串模板”，以“`”符号开头和结尾。在此模板中可以方便地插入变量，使用`\${}`来进行插值。

2.1 Vue实例

■ 方法属性

- Vue的“响应式系统” (Reactivity System)，它是Vue实现很多“魔法”的关键。
- 当一个Vue实例被创建时，data对象中的所有属性会被自动加入到Vue的响应式系统中。当这些属性值发生改变时，视图将会产生“响应”，即匹配更新为新的值。
- 简单的代码说明

```
// 定义一个最简单的数据模型
```

```
var model = { a: 1 }
```

```
// 将该对象加入到一个Vue实例中
```

```
var vm = new Vue({  
  data: model  
})
```

```
// Vue实例中的字段与data中的字段一致
```

```
vm.a == data.a // => true
```

```
// 修改Vue实例上相应的字段，也会影响到data中的原始数据
```

```
vm.a = 2
```

```
console.log(data.a) // => 2
```

```
// 反之，修改data中的字段，也会影响到Vue实例中数据
```

```
data.a = 3
```

```
vm.a // => 3
```

2.1 Vue实例

■ 插入html片段

- 双大括号语法，可以向HTML元素中插入文本；但是如果插入内容不是单纯的文本内容，而是带有HTML结构的内容，就要改用v-html指令了。

```
<div id="app">
  <p v-html="sayHello()"></p>
  <ul>
    <li>姓名 : {{name}}</li>
    <li>城市 : {{city}}</li>
  </ul>
</div>
```

```
methods:{
  sayHello(){
    return `您好，欢迎来自 <b>${this.city}</b> 的 <b>${this.name}</b> !`
  }
}
```

2.2 Vue实例的生命周期

- Vue.js会自动维护每个Vue实例的生命周期，即一个Vue实例，都会经历一系列的从创建到销毁的过程。
- 如创建实例对象、编译模板、将实例挂载到页面上、以及最终的销毁等。在这个过程中，Vue实例在不同阶段的时间点向外部暴露出各自的回调函数，这些回调函数称为“钩子函数”，开发人员可以在这些不同阶段的钩子函数中定义业务逻辑。

■ 生命周期

- 又名生命周期回调函数、生命周期函数、生命周期钩子。
- Vue在关键时刻帮我们调用的一些特殊名称的函数。
- 生命周期函数的名字不可更改，但函数的具体内容是程序员根据需求编写的。
- 生命周期函数中的this指向的是VM或组件实例对象。

```
<body>
  <!-- ...
  <!-- 准备好一个容器-->
  <div id="root">
    <h2 :style="{opacity}">欢迎学习Vue</h2>
    {{ change() }}
  </div>
</body>

<script type="text/javascript">
  Vue.config.productionTip = false //阻止 vue 在启动时生成生产提示。

  new Vue({
    el: '#root',
    data: {
      opacity: 1
    },
    methods: {
      change() {
        setInterval(() => {
          this.opacity -= 0.01
          if (this.opacity <= 0) this.opacity = 1
        }, 16)
      }
    },
  })
</script>
```

欢迎学习Vue

概念引入

```
<script type="text/javascript">
  Vue.config.productionTip = false
```

```
<body>
  <!-- ...
  <!-- 准备好一个容器-->
  <div id="root">
    <h2 v-if="a">你好啊</h2>
    <h2 :style="{opacity}">欢迎学习Vue</h2>
  </div>
</body>
```

```
new Vue({
  el: '#root',
  data: {
    opacity: 1
  },
  methods: {
```

},
//Vue完成模板的解析并把初始的真实DOM元素放入页面后（挂载完毕）调用mounted

```
mounted(){
  console.log('mounted')
  setInterval(() => {
    vm.opacity -= 0.01
    if(vm.opacity <= 0) vm.opacity = 1
  }, 16)
}
```

2.2 Vue实例的生命周期

```
<script>
  let user = {
    name : '',
    city : ''
  };
  let vm = new Vue({
    el: '#app',
    data: user,
    mounted(){
      user = getUserFromApi();
    },
    methods:{
      sayHello(){
        return `您好, 欢迎来自 <b>${this.city}</b>`
      }
    }
  })
</script>
```

- 代码中，定义user对象的时候，name和city两个字段都初始化为空字符串。然后创建Vue实例的时候，在mounted()方法中，调用一个方法获取user对象的属性值。
- mounted()钩子函数是最常用的一个钩子函数，因为是在DOM文档已经渲染完毕以后调用，相当于JavaScript中的window.onload()方法。

Vue完成模板的解析，并把初始的真实DOM元素放入页面后（挂载完毕），调用mounted。

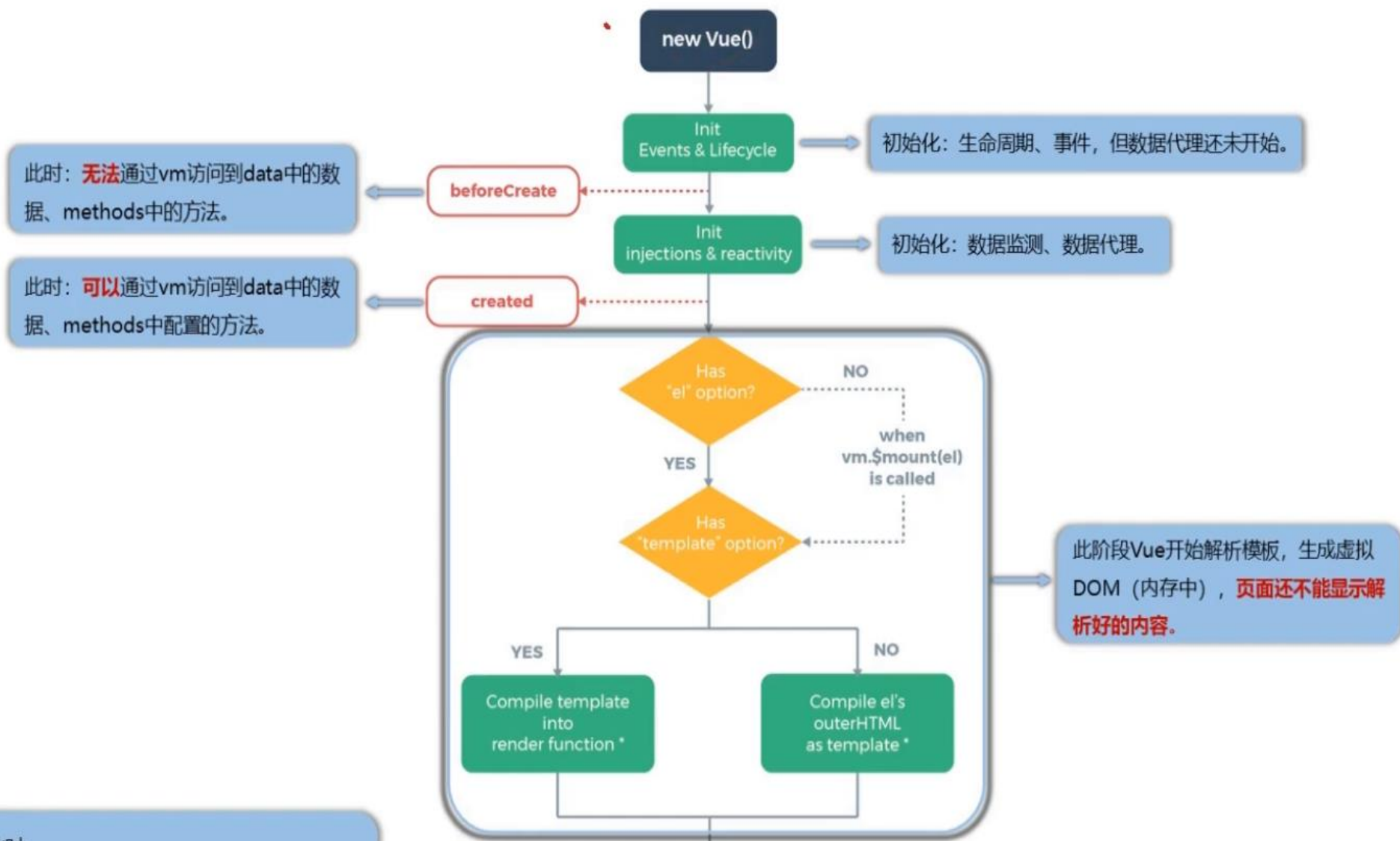
2.2 Vue实例的生命周期

- Vue.js为开发人员提供了众多的生命周期钩子函数，重点理解每个钩子函数在什么时间点被调用。

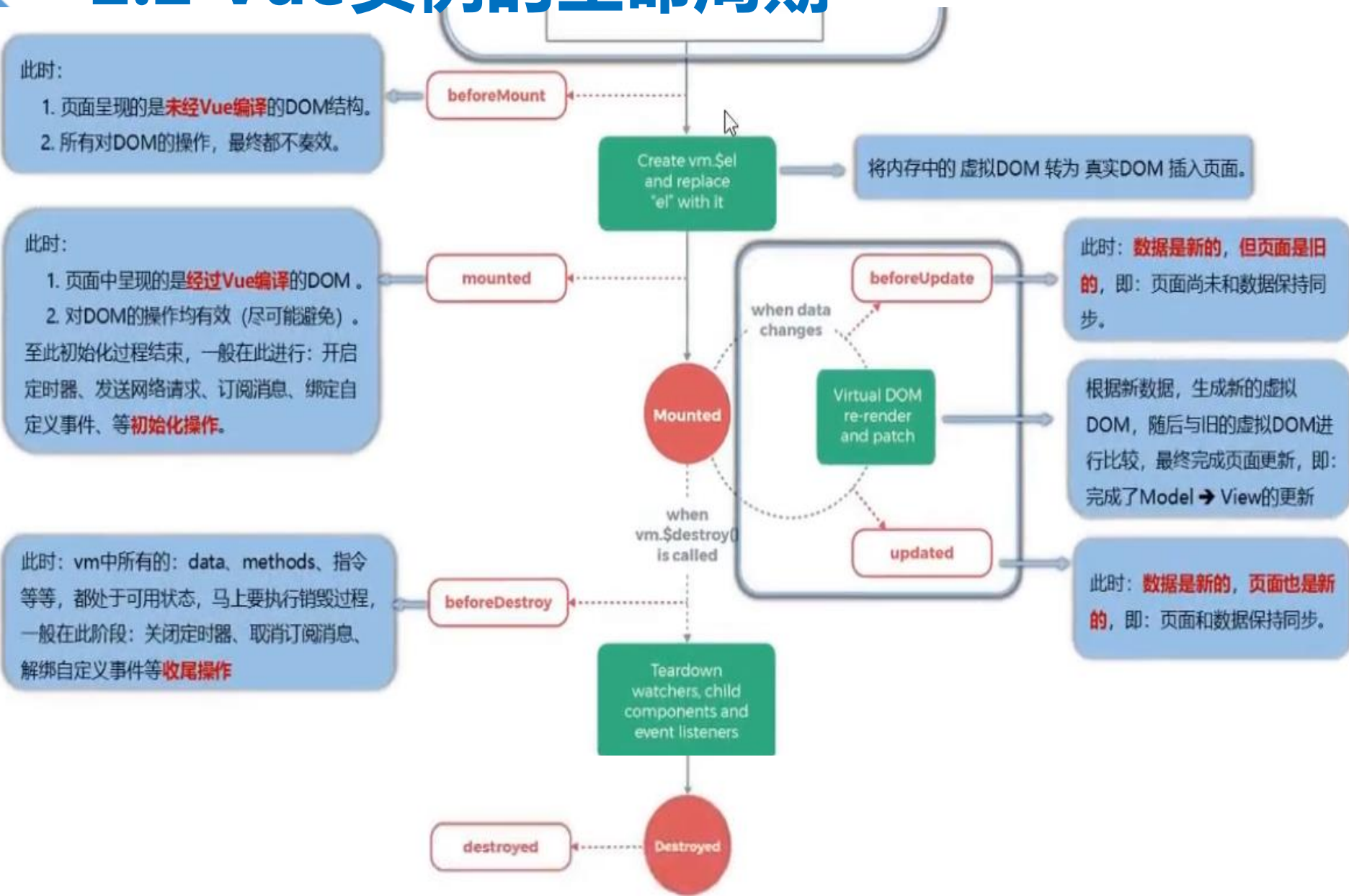
- 钩子函数

beforeCreate	在实例创建之前进行调用
created	在实例创建之后进行调用，此时尚未开始DOM编译。
beforeMount	在挂载开始之前被调用
mounted	实例被挂载后调用，这时页面的相关DOM节点被新创建的vm.\$el替换了。相当于JavaScript中的window.onload()方法。
beforeUpdate	每次页面中有元素需要更新时，在更新前会调用beforeUpdate钩子函数。
updated	每次页面中有元素需要更新时，在更新完成后会调用updated钩子函数。
beforeDestroy	在销毁实例前进行调用，此时实例仍然有效。
destroyed	在实例被销毁之后进行调用。

2.2 Vue实例的生命周期



2.2 Vue实例的生命周期



小结

